# Encapsulation, Inheritance, and Polymorphism

## Theory, Edge Cases & Code Examples

- ✓ Data Hiding and Controlled Access
- ✓ Code Reuse and Hierarchical Relationships
- ✓ Runtime Dynamic Behavior
- ✓ Function Hiding in Inheritance
- ✓ Object Slicing in Polymorphic Hierarchies

→ *Swipe to learn* →

# Encapsulation, Inheritance, and Polymorphism

## Theory

## 📌 1. Encapsulation - Data Hiding and Controlled Access

**Definition:** Encapsulation is the OOP principle of bundling data and methods into a single unit (class) while restricting direct access to internal state, enforced through access specifiers (`private`, `protected`, `public`).

**Core Characteristics:**

| Aspect | Description | Implementation |
|---|---|---|
| **Data hiding** | Internal state invisible to external code | `private` data members |
| **Interface exposure** | Only designated methods access data | `public` member functions |
| **Compile-time enforcement** | Compiler checks access violations | Not runtime security |
| **Invariant protection** | Prevents invalid state | Validation in setters |
| **Friend mechanism** | Legal selective access bypass | `friend` keyword when needed |

**Encapsulation Benefits:**

| Benefit | Example | Impact |
|---|---|---|
| **Maintainability** | Change internal `double` to `BigDecimal` | No client code breaks |
| **Validation** | Ensure balance never goes negative | Prevent invalid states |
| **Loose coupling** | Clients depend on interface only | Implementation flexibility |
| **Security** | Hide sensitive data | Compile-time protection |

**Key Insight:** Encapsulation is compile-time only - memory can be accessed via unsafe pointer casts (UB), so it's not runtime security.

---

# Encapsulation, Inheritance, and Polymorphism

## Theory

## 📌 2. Inheritance - Code Reuse and Hierarchical Relationships

**Definition:** Inheritance allows a derived class to acquire properties and behaviors from a base class, creating hierarchical relationships and enabling polymorphism.

**Inheritance Modes:**

| Mode | Syntax | Base Public → Derived | Base Protected → Derived | Use Case |
|---|---|---|---|---|
| **Public** | `: public Base` | public | protected | "is-a" relationship (polymorphism) |
| **Protected** | `: protected Base` | protected | protected | Restricted inheritance hierarchy |
| **Private** | `: private Base` | private | private | "implemented-in-terms-of" (hide base) |

**Memory Layout:**

- Derived objects contain a complete base class subobject
- Members laid out base-to-derived order in memory
- Base class constructor called first, destructor called last
- Each level adds its own members after the previous level

**Inheritance Principles:**

- **Inheritance mode acts as access ceiling:** Can only restrict, never expand access
- **Private members never accessible in derived:** They exist in memory but cannot be named
- **Substitutability:** Public inheritance enables Liskov Substitution Principle

---

# Encapsulation, Inheritance, and Polymorphism

## Theory

## 📌 3. Polymorphism - Runtime Dynamic Behavior (1/2)

**Definition:** Polymorphism allows objects to be treated as instances of their base class while exhibiting derived class behavior at runtime, implemented via virtual functions, vtables, and vptrs.

### Two Types of Polymorphism:

| Type | Mechanism | Resolution Time | Example |
|---|---|---|---|
| **Compile-time** | Function overloading, templates | Compile time | `void func(int)` vs `void func(double)` |
| **Runtime** | Virtual functions + vtable | Runtime | `virtual void draw()` overridden in derived classes |

### Runtime Polymorphism Implementation:

| Component | Description | Memory Location |
|---|---|---|
| **Virtual function** | Function marked with `virtual` keyword | Code segment |
| **Vtable** | Per-class table of function pointers | Static/Read-only memory (one per class) |
| **Vptr** | Hidden pointer to class's vtable | First member of each object (typically) |
| **Dynamic dispatch** | vptr→vtable[index] lookup at runtime | Runtime indirection |

# Encapsulation, Inheritance, and Polymorphism

## Theory

## 📌 3. Polymorphism - Runtime Dynamic Behavior (2/2)

### How Virtual Dispatch Works:

```
class Base {
public:
    virtual void foo() { std::cout << "Base::foo\n"; }
};

class Derived : public Base {
public:
    void foo() override { std::cout << "Derived::foo\n"; }
};

Base* ptr = new Derived();
ptr->foo();  // Runtime: ptr->vptr->vtable[0]() → Derived::foo
```

### Step-by-step:

1. `Derived` object created with vptr pointing to `Derived`'s vtable
2. Pointer `ptr` of type `Base*` points to the `Derived` object
3. Call `ptr->foo()` dereferences vptr to find vtable
4. Looks up `foo()` entry in `Derived`'s vtable
5. Calls `Derived::foo()` even though pointer is `Base*`

### Virtual Function Overhead:

| Cost Type | Impact | Typical Size |
|---|---|---|
| **Memory per object** | One vptr per object with virtual functions | 8 bytes (64-bit) |
| **Memory per class** | One vtable per class (shared by all objects) | N × 8 bytes (N = virtual function count) |
| **Performance** | One extra indirection per virtual call | ~1-2 CPU cycles |

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 1. Function Hiding in Inheritance

When a derived class defines a function with the same name as a base class function (regardless of signature), it hides all base class overloads of that name. This is called function hiding or name hiding, and it's different from overriding. Even if the base class has multiple overloads of a function, defining any function with that name in the derived class makes all base class versions inaccessible unless explicitly brought into scope.

```
class Base {
public:
    void func(int x) { std::cout << "Base::func(int)\n"; }
    void func(double x) { std::cout << "Base::func(double)\n"; }
};

class Derived : public Base {
public:
    void func(double x) { std::cout << "Derived::func(double)\n"; }
};

int main() {
    Derived d;
    d.func(10);    // ✅ Calls Derived::func(double), not Base::func(int)
    // d.func(5);  // ❌ Base::func(int) is hidden
}
```

This occurs because name lookup stops at the first scope where the name is found. Once `func` is found in `Derived`, the compiler doesn't look in `Base`. To fix this, use `using Base::func;` in the derived class to bring base class overloads into scope.

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 2. Object Slicing in Polymorphic Hierarchies

Object slicing occurs when a derived class object is assigned or passed by value to a base class object. The derived portion is "sliced off," leaving only the base class subobject. This destroys polymorphic behavior because the vtable pointer is replaced with the base class vtable, and all derived class data members are lost.

```
class Animal {
public:
    int age = 0;
    virtual void speak() const { std::cout << "Animal speaks\n"; }
};

class Dog : public Animal {
public:
    int barkVolume = 5;
    void speak() const override { std::cout << "Dog barks\n"; }
};

void printAnimal(Animal a) {  // ❌ Pass by value causes slicing
    a.speak();  // Calls Animal::speak, not Dog::speak
}

int main() {
    Dog d;
    d.age = 3;
    d.barkVolume = 10;
    printAnimal(d);  // Output: "Animal speaks"
}
```

Object slicing is one of the most common polymorphism pitfalls. Always pass polymorphic objects by reference or pointer to preserve their dynamic type and enable correct virtual dispatch.

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 3. Virtual Functions in Constructors and Destructors

Calling virtual functions inside constructors or destructors does not produce polymorphic behavior. During base class construction, the derived part of the object doesn't exist yet, so the vptr points to the base class vtable. Similarly, during destruction, the derived part is destroyed first, so the vptr is updated to point to the base class vtable before the base destructor runs.

```
class Base {
public:
    Base() {
        speak();  // ❌ Calls Base::speak, never Derived::speak
    }
    virtual void speak() { std::cout << "Base speaks\n"; }
};

class Derived : public Base {
public:
    void speak() override { std::cout << "Derived speaks\n"; }
};

int main() {
    Derived d;  // Output: "Base speaks"
}
```

This is a deliberate design choice for safety—if virtual dispatch worked in constructors, calling a derived function before the derived members are initialized could access uninitialized memory. Never rely on virtual dispatch in constructors or destructors.

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 4. Non-Virtual Destructor in Polymorphic Base Classes

If a polymorphic base class (one with virtual functions) has a non-virtual destructor, deleting a derived object through a base pointer causes undefined behavior. Only the base class destructor is called, meaning derived class resources are not cleaned up, leading to resource leaks and potentially corrupted state.

```
class Base {
public:
    ~Base() { std::cout << "Base dtor\n"; }  // ❌ Not virtual
};

class Derived : public Base {
private:
    int* data;
public:
    Derived() : data(new int[100]) {}
    ~Derived() {
        delete[] data;  // ❌ Never called if deleted via Base*
        std::cout << "Derived dtor\n";
    }
};

void destroy(Base* b) {
    delete b;  // ❌ Undefined behavior: only ~Base() runs
}
```

This is such a critical issue that it's considered a fundamental rule: any class with virtual functions must have a virtual destructor. This ensures the correct destruction chain is followed when deleting through a base pointer.

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 5. Access Specifiers and Virtual Function Overriding

Access specifiers control who can call a function, but they do not affect whether a function can be overridden or participate in virtual dispatch. Even private virtual functions are placed in the vtable and can be overridden by derived classes (if the derived class has access to declare the override). The access level only determines calling permissions.

```
class Base {
private:
    virtual void secret() { std::cout << "Base::secret\n"; }
public:
    void callSecret() { secret(); }  // Uses vtable lookup
};

class Derived : public Base {
private:
    void secret() override { std::cout << "Derived::secret\n"; }  // ✅ Legal
};

int main() {
    Derived d;
    d.callSecret();  // Output: "Derived::secret"
}
```

This demonstrates the separation of concerns: access control is a compile-time interface contract, while virtual dispatch is a runtime polymorphism mechanism. They operate independently.

# Encapsulation, Inheritance, and Polymorphism

## Edge Cases

### 📌 6. Default Arguments and Virtual Functions

Default arguments are resolved statically at compile-time based on the static type of the pointer or reference, not the dynamic type of the object. This means that even though virtual function calls use dynamic dispatch, their default arguments do not—they're bound to the type of the pointer used to call the function.

```
class Base {
public:
    virtual void display(int x = 10) {
        std::cout << "Base: " << x << "\n";
    }
};

class Derived : public Base {
public:
    void display(int x = 20) override {
        std::cout << "Derived: " << x << "\n";
    }
};

int main() {
    Base* ptr = new Derived();
    ptr->display();  // Output: "Derived: 10" (not 20!)
}
```

The function call resolves to `Derived::display()` via virtual dispatch, but the default argument `10` comes from the static type `Base*`. This confusing behavior is why many coding standards recommend avoiding default arguments in virtual functions entirely.

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 1. Basic Encapsulation with Private Data

This demonstrates proper encapsulation: the balance cannot be directly manipulated, all access goes through public methods that enforce validation rules, and internal helpers remain private. This ensures that invariants (like non-negative balance) are maintained.

```
class BankAccount {
private:
    double balance;
    std::string accountNumber;

    bool validateAmount(double amount) {
        return amount > 0;  // ✅ Private helper validation
    }

public:
    BankAccount(std::string accNum) : balance(0), accountNumber(accNum) {}

    bool deposit(double amount) {
        if (!validateAmount(amount)) return false;
        balance += amount;  // ✅ Controlled access
        return true;
    }

    bool withdraw(double amount) {
        if (!validateAmount(amount) || amount > balance) return false;
        balance -= amount;
        return true;
    }

    double getBalance() const { return balance; }
};
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 2. Function Hiding and Using-Declaration

Without the `using` declaration, adding any `process` function in `Derived` would hide all base class `process` overloads. The using-declaration makes all base overloads visible alongside the derived class additions.

```
class Base {
public:
    void process(int x) { std::cout << "Base::process(int): " << x << "\n"; }
    void process(double x) { std::cout << "Base::process(double): " << x << "\n"; }
};

class Derived : public Base {
public:
    using Base::process;  // ✅ Brings base overloads into scope
    void process(std::string s) {
        std::cout << "Derived::process(string): " << s << "\n";
    }
};

int main() {
    Derived d;
    d.process(10);          // ✅ Calls Base::process(int)
    d.process(3.14);        // ✅ Calls Base::process(double)
    d.process("hello");     // ✅ Calls Derived::process(string)
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 3. Correct Polymorphic Design with Virtual Destructor

This shows the correct pattern for polymorphic hierarchies: virtual destructor in the base class, pure virtual functions for the interface, and proper override in derived classes. The virtual destructor ensures both `~Circle()` and `~Shape()` are called when deleting through the base pointer.

```
class Shape {
private:
    std::string name;
public:
    Shape(std::string n) : name(n) {}
    virtual ~Shape() { std::cout << "~Shape\n"; }  // ✅ Virtual destructor

    virtual double area() const = 0;  // Pure virtual
    virtual void draw() const = 0;

    std::string getName() const { return name; }
};

class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : Shape("Circle"), radius(r) {}
    ~Circle() override { std::cout << "~Circle\n"; }

    double area() const override { return 3.14159 * radius * radius; }
    void draw() const override { std::cout << "Drawing circle\n"; }
};

int main() {
    Shape* s = new Circle(5.0);
    std::cout << s->area() << "\n";
    delete s;  // ✅ Correct: both destructors called
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 4. Object Slicing Prevention

By making the copy constructor protected, we prevent accidental slicing while still allowing derived classes to implement copying if needed. Passing by reference ensures polymorphic behavior is preserved.

```
class Animal {
protected:
    Animal(const Animal&) = default;  // ✅ Protected copy constructor
    Animal& operator=(const Animal&) = default;
public:
    Animal() = default;
    virtual ~Animal() = default;
    virtual void speak() const = 0;
};

class Dog : public Animal {
public:
    void speak() const override { std::cout << "Woof!\n"; }
};

// Correct way: pass by reference
void makeSpeak(const Animal& a) {  // ✅ Reference, no slicing
    a.speak();
}

int main() {
    Dog d;
    makeSpeak(d);  // ✅ Polymorphism preserved

    // Animal a = d;  // ❌ Would not compile (protected copy)
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 5. Private Virtual Functions and Template Method Pattern

This demonstrates the Template Method pattern: the public interface calls private virtual functions that can be customized by derived classes. The base class controls the algorithm structure while allowing customization of specific steps.

```
class DataProcessor {
private:
    virtual void preProcess() { std::cout << "Default pre-processing\n"; }
    virtual void process() = 0;  // Must override
    virtual void postProcess() { std::cout << "Default post-processing\n"; }

public:
    void execute() {  // ✅ Public non-virtual interface
        preProcess();
        process();
        postProcess();
    }
    virtual ~DataProcessor() = default;
};

class CSVProcessor : public DataProcessor {
private:
    void preProcess() override { std::cout << "CSV: Opening file\n"; }
    void process() override { std::cout << "CSV: Parsing data\n"; }
    // Uses default postProcess
};

int main() {
    DataProcessor* proc = new CSVProcessor();
    proc->execute();  // ✅ Calls overridden private virtuals
    delete proc;
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 6. Multiple Inheritance and vtable Layout

With multiple inheritance, the object has multiple vptrs—one for each base class with virtual functions. Pointer conversions adjust the pointer value to point to the correct subobject, ensuring virtual dispatch works correctly for each interface.

```
class Printable {
public:
    virtual void print() const = 0;
    virtual ~Printable() = default;
};

class Serializable {
public:
    virtual void serialize() const = 0;
    virtual ~Serializable() = default;
};

class Document : public Printable, public Serializable {
private:
    std::string content;
public:
    Document(std::string c) : content(c) {}

    void print() const override {
        std::cout << "Printing: " << content << "\n";
    }

    void serialize() const override {
        std::cout << "Serializing: " << content << "\n";
    }
};

int main() {
    Document doc("Hello");
    Printable* p = &doc;
    Serializable* s = &doc;

    p->print();       // ✅ Virtual dispatch works
    s->serialize();   // ✅ Virtual dispatch works
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 7. Const Correctness in Overriding

The `const` qualifier is part of the function signature for member functions. Omitting it creates a different function that doesn't override the base version. Always use the `override` keyword to catch such mismatches at compile time.

```
class Base {
public:
    virtual void display() const {
        std::cout << "Base const version\n";
    }

    virtual void modify() {
        std::cout << "Base non-const version\n";
    }
};

class Derived : public Base {
public:
    void display() const override {  // ✅ Const matches
        std::cout << "Derived const version\n";
    }

    void modify() override {  // ✅ Non-const matches
        std::cout << "Derived non-const version\n";
    }

    // void display() override {}  // ❌ Would not compile - missing const
};

int main() {
    const Derived d;
    d.display();  // Calls const version

    Derived d2;
    d2.modify();  // Calls non-const version
}
```

# Encapsulation, Inheritance, and Polymorphism

## Code Examples

### 📌 8. Pure Virtual Functions with Default Implementations

A pure virtual function can have an implementation, which derived classes can call explicitly. This is useful for providing common functionality that derived classes can reuse or extend.

```
class Logger {
public:
    virtual void log(const std::string& msg) = 0;  // Pure virtual
    virtual ~Logger() = default;
};

// Provide default implementation
void Logger::log(const std::string& msg) {
    std::cout << "[DEFAULT] " << msg << "\n";
}

class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        Logger::log(msg);  // ✅ Call base implementation
        std::cout << "[FILE] Writing to file\n";
    }
};

int main() {
    Logger* logger = new FileLogger();
    logger->log("System started");
    delete logger;
}
```

# What You Learned

## Key Takeaways

- ✓ Data Hiding and Controlled Access
- ✓ Code Reuse and Hierarchical Relationships
- ✓ Runtime Dynamic Behavior
- ✓ Function Hiding in Inheritance

→ *Swipe to learn* →